UTEM

Implementación de una red neuronal desde cero

Informe técnico

Gianfranco Giannoni

Herramientas para Ciencias de Datos (INFB6052)
Docente: Michael Miranda
Universidad Tecnológica Metropolitana

10 de noviembre de 2026

Índice

1. Resumen **2**

2. Objetivos **2**

3. Problema y datos **2**

4. Metodología **2**

4.1. Arquitectura 2

4.2. Función de pérdida 2

4.3. Retropropagación y actualización 3

4.4. Configuración 3

5. Resultados **3**

6. Conclusiones **4**

1. Resumen

Este informe describe la implementación de una red neuronal artificial desde cero utilizando Python y NumPy, sin recurrir a frameworks de aprendizaje profundo. El modelo corresponde a un perceptrón multicapa con dos entradas, una capa oculta de cuatro neuronas y una salida. Se entrenó para resolver el problema XOR mediante propagación hacia adelante, cálculo de pérdida, retropropagación del error y descenso del gradiente. En los cuatro ejemplos utilizados durante el entrenamiento, el modelo obtuvo las cuatro clases esperadas.

2. Objetivos

- Implementar los componentes básicos de una red neuronal utilizando operaciones matriciales.
- Aplicar funciones de activación y una función de pérdida apropiada para clasificación binaria.
- Entrenar la red mediante retropropagación y descenso del gradiente.
- Evaluar el comportamiento del modelo en el problema lógico XOR.

3. Problema y datos

Se utilizó el problema XOR, cuya salida es 1 cuando las dos entradas son distintas y 0 cuando son iguales. El conjunto contiene cuatro combinaciones:

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

Cuadro 1: Tabla de verdad del problema XOR.

Este problema es útil como demostración porque no es linealmente separable: una única neurona sin capa oculta no puede representar correctamente sus cuatro casos.

4. Metodología

4.1. Arquitectura

La red implementada tiene arquitectura 2–4–1: dos variables de entrada, cuatro neuronas en la capa oculta y una neurona de salida. La capa oculta utiliza ReLU y la salida utiliza Sigmoid para producir una probabilidad entre 0 y 1.

$$Z_1 = XW_1 + b_1, \qquad A_1 = \mathrm{ReLU}(Z_1)$$
$$Z_2 = A_1W_2 + b_2, \qquad \hat{Y} = \sigma(Z_2)$$

4.2. Función de pérdida

Se utiliza Binary Cross-Entropy (BCE), que mide la diferencia entre las etiquetas reales y las probabilidades predichas:

$$L = -\frac{1}{m}\sum_{i=1}^{m}\left[y_i \log(\hat{y}_i) + (1-y_i)\log(1-\hat{y}_i)\right].$$

4.3. Retropropagación y actualización

La retropropagación calcula cómo contribuye cada parámetro al error. Para la combinación de Sigmoid en la salida y BCE, el gradiente respecto de la preactivación de salida se simplifica a:

$$dZ_2 = \hat{Y} - Y.$$

Luego se calculan los gradientes de pesos y sesgos de ambas capas y se actualizan mediante descenso del gradiente:

$$\theta_{\text{nuevo}} = \theta_{\text{anterior}} - \alpha \nabla_\theta L,$$

donde α es la tasa de aprendizaje. El entrenamiento utiliza los cuatro ejemplos en cada actualización (full-batch gradient descent).

4.4. Configuración

Se entrenó durante 20 000 épocas, con tasa de aprendizaje 0.1 y cuatro neuronas ocultas. La implementación se distribuye en módulos: `activations.py`, `loss.py`, `neural_network.py` y `main.py`. Las pruebas automatizadas verifican ReLU, Sigmoid, Binary Cross-Entropy y el aprendizaje del patrón XOR.

5. Resultados

La pérdida disminuyó durante el entrenamiento. Las predicciones finales obtenidas fueron:

x_1	x_2	Probabilidad predicha	Clase ($\hat{y} \geq 0.5$)
0	0	0.0025	0
0	1	0.9997	1
1	0	0.9998	1
1	1	0.0002	0

Cuadro 2: Predicciones reportadas para los cuatro ejemplos XOR.

La pérdida registrada en la época 18 000 fue aproximadamente 0.000875. El programa también genera un gráfico de la evolución de la pérdida durante el entrenamiento.

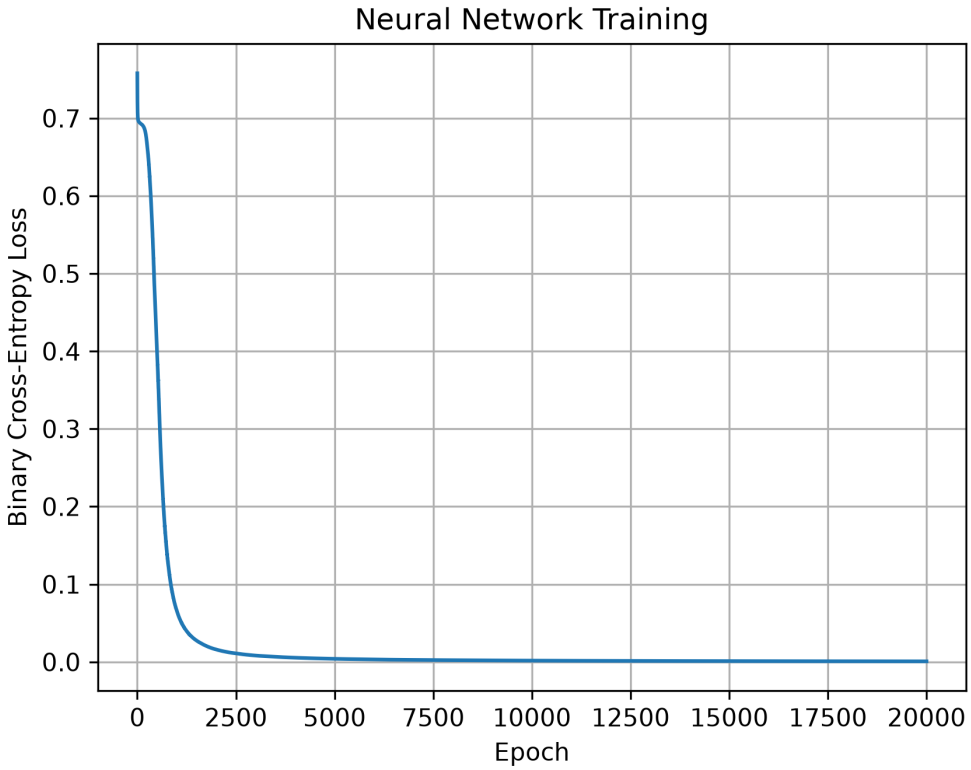


Figura 1: Evolución de la pérdida durante el entrenamiento.

Las cuatro predicciones coinciden con las etiquetas del conjunto de entrenamiento. Esto demuestra que la red aprendió a representar los cuatro casos presentados, pero no constituye una evaluación de generalización, ya que no se utilizó un conjunto de prueba independiente.

6. Conclusiones

Se implementó y entrenó una red neuronal multicapa utilizando Python y NumPy, programando explícitamente la propagación hacia adelante, la función de pérdida, la retropropagación y la actualización de parámetros. El modelo clasificó correctamente los cuatro ejemplos del problema XOR. La experiencia permitió aplicar los componentes fundamentales del aprendizaje supervisado. Como limitación, el conjunto de datos es mínimo y coincide con los ejemplos de entrenamiento; por ello, los resultados no permiten afirmar capacidad de generalización a datos nuevos.

Material complementario

El código fuente y las pruebas se encuentran en el directorio `Red_Neuronal`. Para ejecutar el experimento desde la raíz del repositorio:

```
python Red_Neuronal/src/main.py
python -m unittest discover -s Red_Neuronal/tests -v
```